

ROS2 Communication Optimization Using micro-ROS and XRCE-DDS

Par Guillaume PORET

••• Sommaire

- 01.** Micro-ROS
- 02.** XRCE-DDS
- 03.** Principe de Fonctionnement
- 04.** Exemple Fonctionnel
- 05.** Avantages et Inconvénients

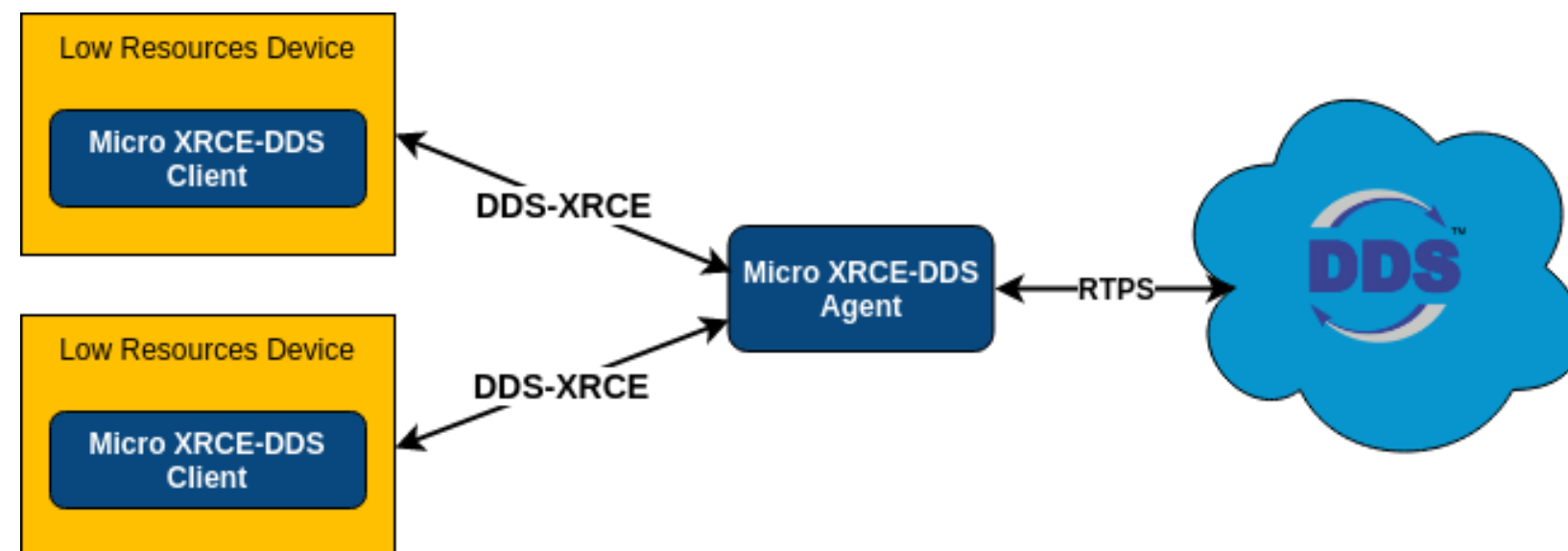
MICRO-ROS

- Micro-ROS est une implémentation de ROS 2 spécifiquement adaptée pour les microcontrôleurs et les systèmes embarqués avec des ressources limitées. Il est conçu pour offrir les avantages de ROS dans des environnements où les ressources matérielles sont restreintes.
- Compatibilité avec ROS 2 : Micro-ROS conserve une compatibilité totale avec ROS 2, permettant aux développeurs de réutiliser des codes et des outils existants dans l'écosystème ROS pour une transition plus douce et moins coûteuse.
- Intégration avec XRCE de ProSima : Micro-ROS intègre déjà les paquets XRCE de ProSima, facilitant la mise en place de communications efficaces et légères adaptées aux environnements à ressources limitées.



02. XRCE-DDS

- XRCE-DDS (eXtremely Resource-Constrained Environments Data Distribution Service) : Un middleware conçu pour optimiser la communication dans des systèmes où les ressources matérielles sont limitées. Il est basé sur le standard DDS (Data Distribution Service), mais adapté pour minimiser l'utilisation de la bande passante et des ressources de calcul.

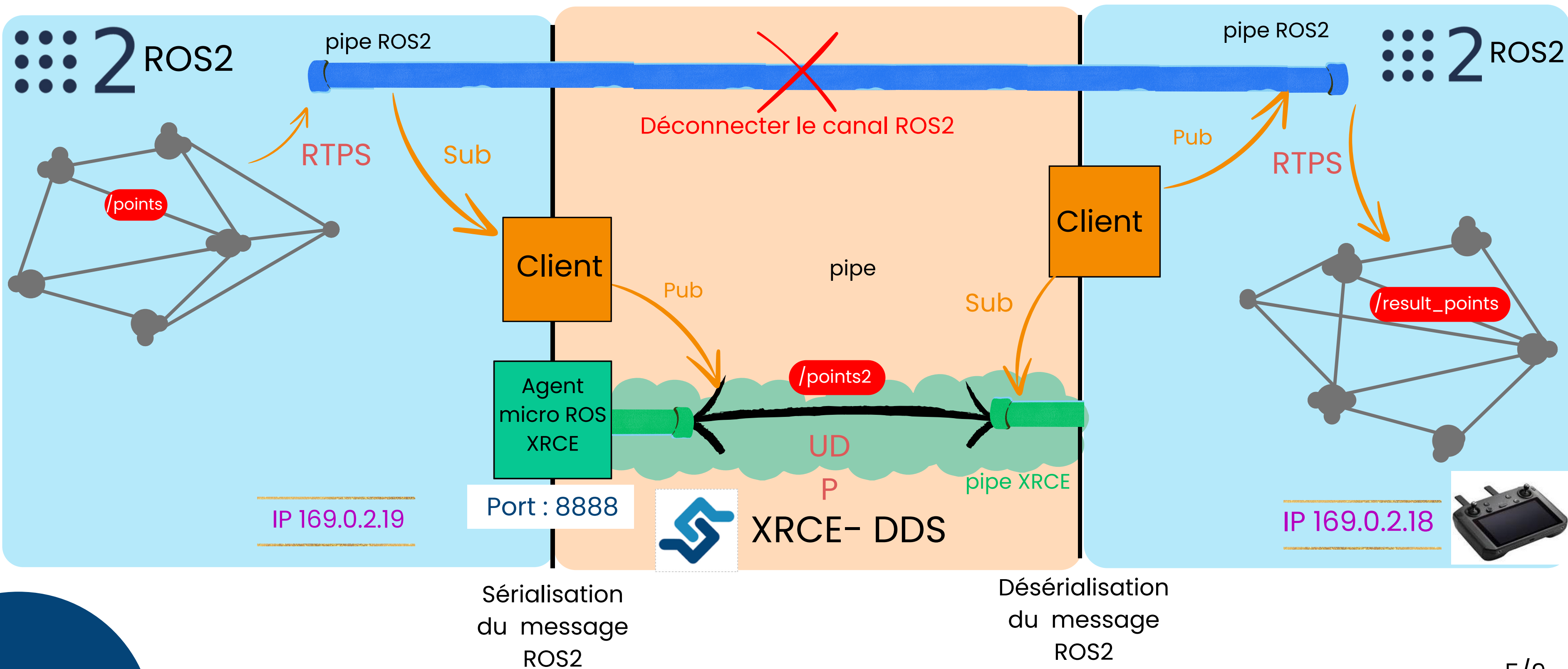


Architecture Client-Agent

Structure : XRCE-DDS fonctionne selon une architecture Client-Agent où le client est un élément léger, exécuté sur un microcontrôleur avec micro-ROS, tandis que l'agent agit comme un intermédiaire entre le client et les autres nœuds ROS 2.

Communication : Le client communique avec l'agent en utilisant un protocole UDP ou TCP, selon la configuration du réseau. L'agent traite ces requêtes et les traduit en actions ROS 2 standards.

03. PRINCIPE DE FONCTIONNEMENT



03. PRINCIPE DE FONCTIONNEMENT

- L'agent XRCE-DDS joue un rôle pivot dans le fonctionnement de notre système en établissant un pont de communication entre deux machines: une télécommande et un robot. L'agent sert d'intermédiaire qui remplace le "pipe" ROS2 traditionnel, permettant une communication plus légère et adaptée aux systèmes embarqués grâce au protocole XRCE, moins gourmand en ressources.
- A chaque extrémité de la chaîne de communication, des clients micro-ROS sont établis. Ces clients fonctionnent sur le robot et sur la télécommande. Chaque client est responsable de la création de nœuds ROS, qui sont des points de communication essentiels pour la transmission des données. Les clients définissent les topics auxquels ils souhaitent souscrire et ceux qu'ils veulent publier via le pipe XRCE.
- Les messages ROS2, une fois générés par la télécommande ou le robot, sont sérialisés, c'est-à-dire convertis en un format de données compact pour la transmission. Cette sérialisation réduit la taille des données, facilitant ainsi leur envoi sur des réseaux à capacité limitée.
- Les messages sérialisés sont envoyés à la télécommande via UDP, un protocole de communication rapide et efficace pour les systèmes temps réel. Une fois reçus par le client micro-ROS de destination, les messages sont désérialisés, c'est-à-dire retransformés dans leur format original pour être utilisés.
- L'installation d'un environnement ROS 2 sur les machines clientes (télécommande) n'est pas obligatoire selon l'utilisation ; cependant, la présence d'un environnement micro-ROS est indispensable pour assurer la connectivité et la communication entre les clients et l'agent.

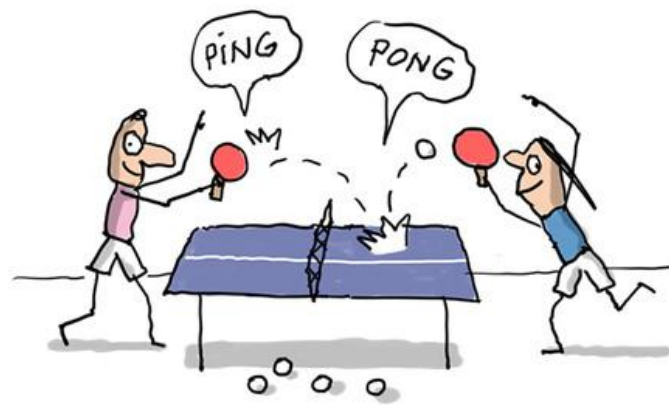
04. EXEMPLE FONCTIONNEL

1. Communication Ping-Pong avec std_msgs

- Objectif : Démontrer la communication bidirectionnelle simple entre deux nœuds micro-ROS.

Fonctionnement :

- Ping : Un nœud envoie un message "ping" à intervalles réguliers en utilisant le type de message std_msgs/msg/Header.
- Pong : Un autre nœud reçoit le message "ping" et répond avec un message "pong"

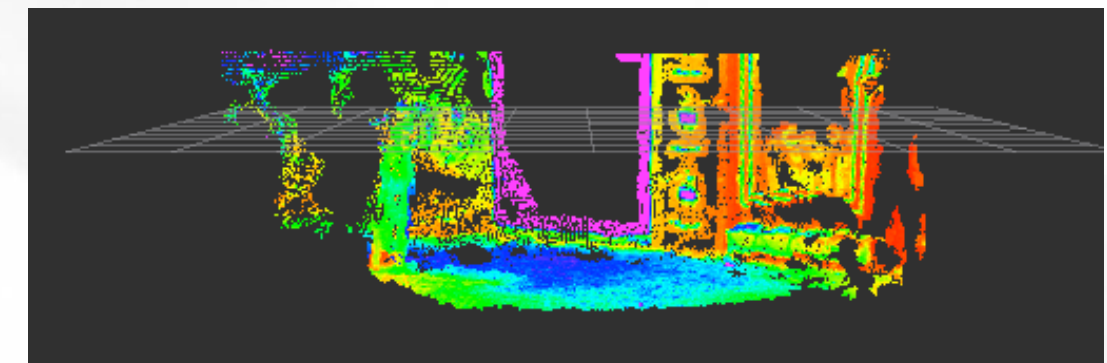


2. Test de Transmission avec rosbag et PointCloud2

- Objectif : Tester et valider la transmission de données complexes, telles que les nuages de points issus d'un lidar.

Fonctionnement :

- Publication : Un nœud micro-ROS publie des données sensor_msgs/msg/PointCloud2 sur un topic ROS2, extraites d'un rosbag contenant une simulation de données lidar.
- Souscription : Un autre nœud souscrit à ce nouveau topic pour recevoir les données du nuage de points et publie sur un topic std_msgs/msg/String certaines valeurs extraites de celles-ci.



04. EXEMPLE FONCTIONNEL

Remarques :

- **Restriction des Communications ROS 2:**

La variable d'environnement `ROS_LOCALHOST_ONLY=1` est utilisée pour limiter les communications des nœuds ROS 2 à la même machine, en restreignant les communications du middleware ROS 2 (DDS) aux interfaces localhost. Une tentative a été faite pour utiliser différents `ROS_DOMAIN_ID` sur chaque machine afin de segmenter les communications entre les systèmes ROS, mais cela a échoué à cause d'un fichier de configuration de micro-ROS qui fixe par défaut le `ROS_DOMAIN_ID` à 0. Ce problème est connu mais reste sans solution clairement définie dans les forums en ligne.

- **Configuration de micro-ROS:**

Certains fichiers de configuration de micro-ROS sont cachés et nécessitent une investigation approfondie pour être modifiés ou utilisés correctement.

- **Connexion Multi-Clients :**

Il est possible de connecter plusieurs clients depuis différentes machines à un seul agent, permettant par exemple l'utilisation de deux télécommandes.

AVANTAGES ET INCONVÉNIENTS

Critère/Protocole	XRCE-DDS	MAVLink	JAUS
Format du Message	Binaire	XML pour la définition, Binaire pour l'échange	Binaire
Compatibilité avec ROS2	Haute (Conçu pour ROS2)	Moyenne (Nécessite des adaptateurs tel que MavRos)	Faible (Nécessite des adaptateurs complexes)
Efficacité	Très haute (Optimisé pour les environnements à ressources limitées)	Haute (Conçu pour les systèmes aériens non habités)	Moyenne à variable (Dépend de l'implémentation) mais reste le plus flexible
Complexité de Mise en Place	Moyenne (Requiert la configuration de l'agent XRCE)	Basse (Protocole bien documenté et largement utilisé dans les UAVs)	Élevée (Protocole complexe avec de nombreux paramètres)
Documentation en Ligne	Correcte (Support étendu par la communauté ROS et DDS)	Très abondante (Large utilisation dans la communauté drone)	Limitée (Moins répandue hors des applications militaires ou similaires)
Interopérabilité	Excellente avec les systèmes ROS2	Très Bonne avec les systèmes aériens non habités (UAVs) mais moyenne pour les drones terrestres et maritimes	Spécifiquement conçue pour les systèmes sans pilote dans des contextes militaires
Latence	Très basse (Optimisée pour la communication en temps réel)	Basse (Adaptée pour le contrôle en temps réel des UAVs)	Variable (Peut être optimisée mais généralement pas aussi basse que XRCE ou MAVLink)
Sécurité	Assez élevée (Supporte diverses mesures de sécurité DDS)	Modérée (Des extensions sont nécessaires pour une sécurité renforcée)	Très Élevée (Conçu avec des considérations de sécurité militaire)